

# 中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并行程序设计

批改人：

|       |                      |        |           |
|-------|----------------------|--------|-----------|
| 实验    | Pthreads 并行矩阵乘法与数组求和 | 专业（方向） | 计算机科学与技术  |
| 学号    | 22320021             | 姓名     | 陈安康       |
| Email | 1743826979@qq.com    | 完成日期   | 2025/4/13 |

## 1. 实验目的

熟悉线程并行化的理论并进行实践，利用 Pthread 实现并行矩阵乘法与数组求和，并对结果进行分析与讨论。学习 Pthread 的理论以及具体的实践方法。

## 2. 实验过程和核心代码

### 任务 1

利用 Pthread 库进行并行矩阵乘法实现。不同于 MPI 库的进程并行，线程并行过程中所有的内存都是共享的，因此可以直接对原矩阵进行操作而不用对矩阵进行部分分发。核心实现代码如下，对矩阵进行按行分块处理，使用 pthread\_create 创建多个线程，通过自定义的结构体传递参数，在主线程中使用 pthread\_join 函数等待所有计算完成后即可得到结果。

```

void* matrixMultiplyThread(void *arg) {
    ThreadData *data = (ThreadData*)arg;
    int chunk_size = data->m / data->thread_count;
    int start_row = data->thread_id * chunk_size;
    int end_row = (data->thread_id == data->thread_count - 1) ? data->m : start_row + chunk_size;

    for (int i = start_row; i < end_row; ++i) {
        for (int j = 0; j < data->k; ++j) {
            double sum = 0.0;
            for (int p = 0; p < data->n; ++p) {
                sum += data->A[i * data->n + p] * data->B[p * data->k + j];
            }
            data->C[i * data->k + j] = sum;
        }
    }
    return NULL;
}

```

```

int main(int argc, char **argv) {
    if (argc != 5) {
        printf("Usage: %s m n k thread_count\n", argv[0]);
        return 1;
    }

    int m = atoi(argv[1]);
    int n = atoi(argv[2]);
    int k = atoi(argv[3]);
    long thread_count = atol(argv[4]);

    double *A = malloc(sizeof(double) * m * n);
    double *B = malloc(sizeof(double) * n * k);
    double *C = malloc(sizeof(double) * m * k);

    fillMatrix(A, m, n);
    fillMatrix(B, n, k);

    pthread_t* threads = malloc(thread_count * sizeof(pthread_t));
    ThreadData thread_data[thread_count];

    double start = getTimeInSeconds();

    for (int i = 0; i < thread_count; ++i) {
        thread_data[i] = (ThreadData){i, thread_count, m, n, k, A, B, C};
        pthread_create(&threads[i], NULL, matrixMultiplyThread, &thread_data[i]);
    }

    for (int i = 0; i < thread_count; ++i) {
        pthread_join(threads[i], NULL);
    }

    double end = getTimeInSeconds();
    double elapsed = end - start;
}

```

## 任务 2

利用 pthread 实现并行数组求和。这里考虑不同的实现方式，主要是聚合方式的不同。

第一种是全局变量+互斥锁。通过线程将数组每一部分分别求和，然后将其累加到一个全局变量中。为了解决共享资源访问时的同步问题，设置互斥锁保证一次只能由一个线程访问全局变量。实现核心代码如下：

```
void* partial_sum(void* arg) {
    ThreadData *data = (ThreadData*)arg;
    long long local_sum = 0;
    for (long i = data->start; i < data->end; ++i)
        local_sum += A[i];

    pthread_mutex_lock(&sum_lock);
    global_sum += local_sum;
    pthread_mutex_unlock(&sum_lock);
    return NULL;
}
```

```

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <array_size_in_M> <thread_count>\n", argv[0]);
        return 1;
    }

    long N = atol(argv[1]) * 1000 * 1000;
    // long N = atol(argv[1]);
    int thread_count = atoi(argv[2]);
    pthread_t* threads = malloc(thread_count * sizeof(pthread_t));
    ThreadData thread_data[thread_count];

    A = malloc(sizeof(int) * N);
    srand((unsigned int)time(NULL));
    for (long i = 0; i < N; ++i) A[i] = rand() % 100;

    for (int i = 0; i < N; i++) {
        printf("%d ", A[i]);
    }

    pthread_mutex_init(&sum_lock, NULL);

    double start_time = get_time();

    long chunk = N / thread_count;
    for (int i = 0; i < thread_count; ++i) {
        thread_data[i].thread_id = i;
        thread_data[i].start = i * chunk;
        thread_data[i].end = (i == thread_count - 1) ? N : (i + 1) * chunk;
        pthread_create(&threads[i], NULL, partial_sum, &thread_data[i]);
    }

    for (int i = 0; i < thread_count; ++i)
        pthread_join(threads[i], NULL);

    double end_time = get_time();
}

```

第二种实现方法是通过一个共享数组来模拟点对点通信的场景，所有的线程将部分和结果放在一个共享数组中，主线程等待所有的线程执行结束后遍历数组将数组中的结果进行累加。实现核心代码如下：

```

void* partial_sum(void* arg) {
    ThreadData *data = (ThreadData*)arg;
    long long local_sum = 0;
    for (long i = data->start; i < data->end; ++i)
        local_sum += A[i];
    local_sums[data->thread_id] = local_sum;
    return NULL;
}

```

```

double start_time = get_time();

long chunk = N / thread_count;
for (int i = 0; i < thread_count; ++i) {
    thread_data[i].thread_id = i;
    thread_data[i].start = i * chunk;
    thread_data[i].end = (i == thread_count - 1) ? N : (i + 1) * chunk;
    pthread_create(&threads[i], NULL, partial_sum, &thread_data[i]);
}

for (int i = 0; i < thread_count; ++i)
    pthread_join(threads[i], NULL);

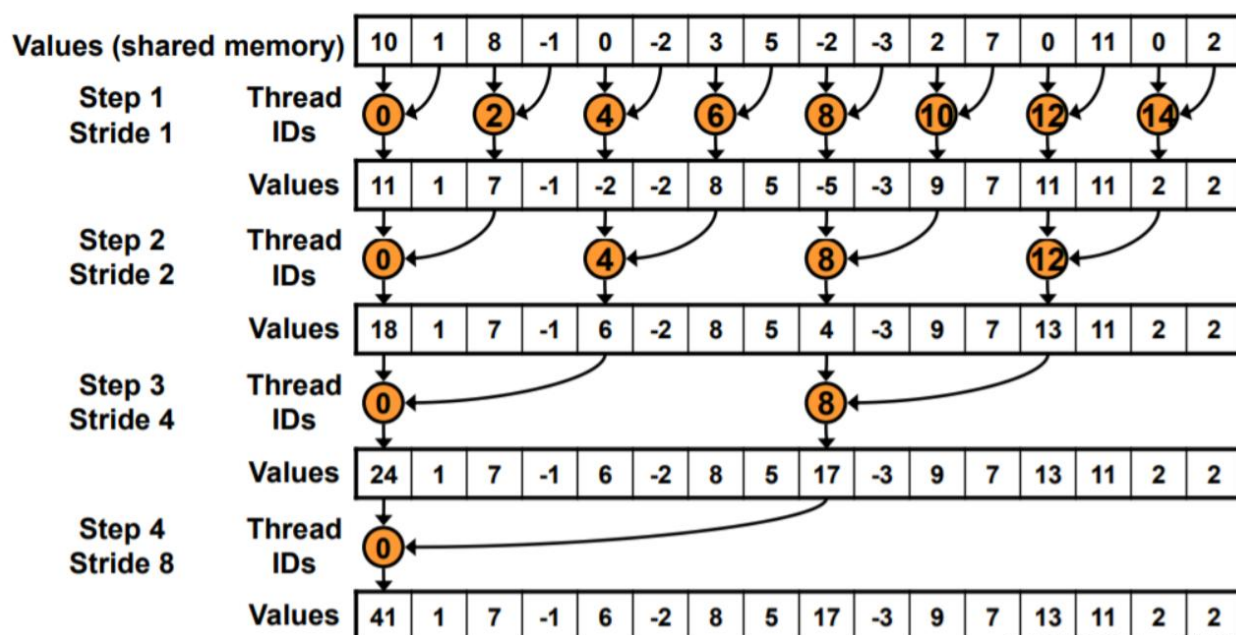
long long total = 0;
for (int i = 0; i < thread_count; ++i)
    total += local_sums[i];

double end_time = get_time();

```

另一种规约方法是在此基础上的改进，实现相邻配对，使  $O(n)$  时间的集合缩短为  $O(\log n)$  时间。具体做法是每次轮回将存放结果的数组中的相邻未合并数据进行相加，并用相加的和更新合并块的头部数据。直到所有数据都进行合并，数组头部即为数组和。整个过程如下图：





将聚合过程也放在线程中并行实现，提升效率。由于需要保证每一轮迭代时所有的进程都同步到此轮迭代中，因此需要实现所有线程的同步。这里有多种方式，一种是 pthread 的现成的屏障（barriers）；还有一种是通过互斥锁和条件变量来实现屏障机制。这里我采用后者，实现代码如下：

```

void* sum_array_reduce(void* arg) {
    ThreadArgs* args = (ThreadArgs*)arg;
    int thread_id = args->thread_id;
    int num_threads = args->num_threads;
    int* array = args->array;
    long long size = args->size;
    long long* partial_sums = args->partial_sums;
    int max_rounds = args->max_rounds;
    pthread_mutex_t* mutex = args->mutex;
    pthread_cond_t* cond = args->cond;
    int* round_completed = args->round_completed;
    long long elements_per_thread = size / num_threads;
    long long start = thread_id * elements_per_thread;
    long long end = (thread_id == num_threads - 1) ? size : start + elements_per_thread;

    long long local_sum = 0;
    for (long long i = start; i < end; i++) {
        local_sum += array[i];
    }
    partial_sums[thread_id] = local_sum;
    // 同步用的, 使用了互斥锁+条件变量
    pthread_mutex_lock(mutex);
    (*round_completed)++;

    if (*round_completed == num_threads) {
        *round_completed = 0;
        pthread_cond_broadcast(cond);
    } else {
        pthread_cond_wait(cond, mutex);
    }
    pthread_mutex_unlock(mutex);
}

```

```

    for (int round = 0; round < max_rounds; round++) {

        int stride = 1 << round;
        if (thread_id % (stride * 2) == 0 && thread_id + stride < num_threads) {
            partial_sums[thread_id] += partial_sums[thread_id + stride];
            // printf("%lld ", partial_sums[thread_id]);
        }

        pthread_mutex_lock(mutex);
        (*round_completed)++;
        if (*round_completed == num_threads) {
            *round_completed = 0;
            pthread_cond_broadcast(cond);
        } else {

            pthread_cond_wait(cond, mutex);
        }
        pthread_mutex_unlock(mutex);
    }

    return NULL;
}

```

### 3. 实验结果

为了方便观察，将矩阵和数组输出的代码都注释掉了

对于任务 1，首先考虑方阵，实验结果如下：

| 进程数 | 矩阵规模     |          |          |          |           |
|-----|----------|----------|----------|----------|-----------|
|     | 128      | 256      | 512      | 1024     | 2048      |
| 1   | 0.004141 | 0.034170 | 0.375929 | 3.727200 | 44.179638 |
| 2   | 0.002402 | 0.017558 | 0.201458 | 1.600374 | 19.792323 |
| 4   | 0.001359 | 0.010987 | 0.096310 | 0.854172 | 10.029392 |
| 8   | 0.001268 | 0.007441 | 0.061955 | 0.559405 | 8.868567  |
| 16  | 0.001007 | 0.007210 | 0.051882 | 0.509023 | 6.766098  |

实验过程部分结果展示：

```

cake@CAK:~/并行程序设计/hw4$ ./matrix_multiply 2048 2048 2048 16
Time elapsed: 6.713862 seconds

```

在这里贴一张前面实验使用 MPI 的结果，两个代码运行在相同的 wsl 上，因此不存在硬件性能差异。



| 进程数 | 矩阵规模   |        |        |        |          |
|-----|--------|--------|--------|--------|----------|
|     | 128    | 256    | 512    | 1024   | 2048     |
| 1   | 0.0044 | 0.0321 | 0.3715 | 3.3194 | 107.0133 |
| 2   | 0.0020 | 0.0172 | 0.1923 | 1.9922 | 50.7277  |
| 4   | 0.0010 | 0.0101 | 0.0976 | 1.2159 | 29.9348  |
| 8   | 0.0005 | 0.0051 | 0.0637 | 0.7949 | 20.8303  |
| 16  | 0.0005 | 0.0047 | 0.0509 | 0.5560 | 13.1309  |

由于在此次实验时我使用的是 wsl，而上次实验使用的是虚拟机，因此在 wsl

对比之前 MPI 的实验结果，可以发现在线程数较多（比较两个表格线程数 8-16 的实验结果），矩阵规模大（矩阵规模为 2048 时的实验结果）时，使用线程比使用进程的性能提升是十分明显的。原因是因为进程创建的代价远大于线程，并且进程的分发和聚合代价较大，而线程由于所有的线程都共享同一份内存，因此可以直接在原矩阵上操作，代价很小。

在可扩展性上，当矩阵规模继续增大，或者并行（线程\*进程）数增大时，对于 MPI，分发所需要的代价都会随之变高，而使用线程却没有这部分代价增长，因此在可扩展性上，当硬件不是瓶颈时，对于目前的实现，使用线程相比于使用进程可扩展性更好。

对于不同的数据，比如瘦长的矩阵或者矮胖的矩阵，前者使用列划分，后者使用行划分，这样缓存命中率更高，性能提升明显。

对于稀疏矩阵，使用其他的存储方式如 CSR 等可以在计算时显著提升性能。

对比三种聚合方式下数组求和的性能：

全局变量+互斥锁：

| 进程数 | 数组规模（M）  |          |          |          |           |
|-----|----------|----------|----------|----------|-----------|
|     | 128      | 256      | 512      | 1024     | 2048      |
| 1   | 0.222500 | 0.443775 | 1.061565 | 2.042983 | 42.236895 |
| 2   | 0.136827 | 0.265132 | 0.526787 | 0.983544 | 20.573444 |
| 4   | 0.060182 | 0.128364 | 0.310252 | 0.557649 | 11.235225 |
| 8   | 0.036687 | 0.067351 | 0.144166 | 0.287428 | 6.162380  |
| 16  | 0.026780 | 0.049750 | 0.076032 | 0.164337 | 4.034396  |

共享数组模拟点对点通信：

| 进程数 | 数组规模（M）  |          |          |          |           |
|-----|----------|----------|----------|----------|-----------|
|     | 128      | 256      | 512      | 1024     | 2048      |
| 1   | 0.220472 | 0.439315 | 0.885468 | 1.828764 | 46.210773 |
| 2   | 0.113146 | 0.229654 | 0.467673 | 0.921379 | 18.210758 |

|    |          |          |          |          |           |
|----|----------|----------|----------|----------|-----------|
| 4  | 0.061770 | 0.124461 | 0.255601 | 0.666301 | 10.006505 |
| 8  | 0.063221 | 0.074907 | 0.141730 | 0.270800 | 6.417290  |
| 16 | 0.032200 | 0.044505 | 0.104106 | 0.169397 | 5.016275  |

#### 相邻匹配的聚合方式：

| 进程数 | 数组规模（M） |       |       |       |        |
|-----|---------|-------|-------|-------|--------|
|     | 128     | 256   | 512   | 1024  | 2048   |
| 1   | 0.222   | 0.440 | 0.874 | 1.749 | 41.694 |
| 2   | 0.112   | 0.236 | 0.459 | 0.908 | 20.830 |
| 4   | 0.061   | 0.129 | 0.244 | 0.500 | 11.366 |
| 8   | 0.032   | 0.061 | 0.146 | 0.239 | 6.367  |
| 16  | 0.045   | 0.044 | 0.105 | 0.167 | 3.669  |

三种聚合方式的差别不大，推测可能得原因是进程数过少，以上的聚合方式都是针对进程数的，当进程数不大时优化效果并不明显。可是如果进程数再增加，由于核数受限，并行转并发，也不会看出任何的性能提升，由此现在对于更多进程的分析也只能停留在理论上。在理论上，点对点与普通规约性能一致，相邻匹配性能要优于两者。

在可扩展性上，考虑线程数增加的情况，三种聚合方式的聚合复杂度分别是  $O(n)$ 、 $O(n)$ 、 $O(\log n)$ 。当硬件设置跟得上时，相邻匹配的性能最佳。

## 4. 实验感想

通过这次实验，我使用 pthread 完成了并行矩阵乘法，并对比 MPI 使用进程实现的并行矩阵乘法，分析两者的性能差异。并且通过 pthread 实现了数组求和，实现了三种不同的聚合方式，并进行分析。此次实验我锻炼了 pthread 的应用能力，受益匪浅。